

Project Report: Multi-Packet GMSK Communication System with CSV Interface

February 15, 2026

1 Introduction

This report documents a complete MATLAB implementation of a GMSK (Gaussian Minimum Shift Keying) communication system supporting multi-packet transmission with AXI4-Stream CSV interface. The system implements a full digital communication pipeline including packet building, convolutional encoding, GMSK modulation, AWGN channel simulation, preamble detection, soft-decision Viterbi decoding, and CSV-based data input/output.

The system is designed to transmit arbitrary-length data payloads across multiple packets, with each packet containing a 34-byte information payload encoded using a rate-1/2 convolutional code. The receiver performs correlation-based preamble detection to identify packet boundaries and recovers the transmitted data through soft-decision Viterbi decoding.

2 System Overview

2.1 Architecture

The system follows a conventional digital communication architecture:

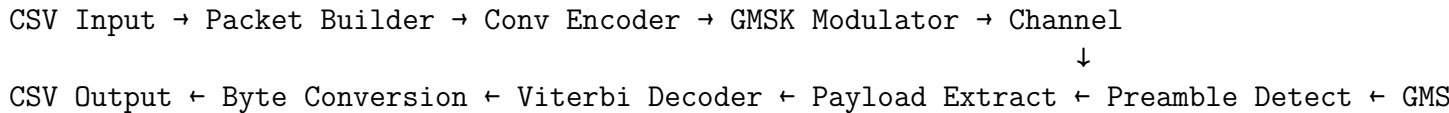


Figure 1: System Block Diagram

2.2 Key Features

- Support for arbitrary-length payload transmission via multi-packet aggregation
- Rate-1/2 convolutional encoding with K=7 constraint length
- Gaussian pulse shaping for spectral efficiency

- Soft-decision Viterbi decoding for improved error correction
- Correlation-based preamble detection for packet synchronization
- AXI4-Stream CSV format for data input/output
- Comprehensive diagnostic plots and error metrics

3 Packet Structure

The Waveform class (waveformId=0) defines a fixed packet structure optimized for robust communication:

3.1 Packet Composition

Each packet contains the following fields:

Field	Length (bits)	Description
Preamble	48	Known synchronization sequence for detection
Payload Chunk 1	136	First quarter of encoded payload (136 bits)
Training Sequence 1	8	First quarter of training sequence
Payload Chunk 2	136	Second quarter of encoded payload
Training Sequence 2	8	Second quarter of training sequence
Payload Chunk 3	136	Third quarter of encoded payload
Training Sequence 3	8	Third quarter of training sequence
Payload Chunk 4	136	Fourth quarter of encoded payload
Training Sequence 4	8	Fourth quarter of training sequence
Total Burst	624	Complete packet length

3.2 Payload Parameters

- Information payload: 34 bytes (272 bits)
- Encoded payload: 544 bits (rate 1/2 convolutional coding)
- Preamble: 48 bits (pseudo-random sequence for correlation detection)
- Training sequences: 32 bits total (8 bits $\times$ 4, interleaved)
- Total burst: 624 bits

The interleaved structure places training sequences between payload chunks to assist with channel estimation and provide known reference points within the burst.

4 Transmitter Components

4.1 Waveform Class (`Waveform.m`)

The Waveform class defines the packet structure parameters:

- `waveformId`: Identifies the waveform configuration (0 for FEC rate 1/2)
- `pldLenInfBytes`: Information payload length in bytes (34)
- `pldLenEncBits`: Encoded payload length in bits (544)
- `prmbSeqLen`: Preamble sequence length (48 bits)
- `trngSeqLen`: Training sequence length (32 bits)
- `burstBlockLen`: Total burst length (624 bits)

The class generates reproducible preamble and training sequences using a fixed random seed (12345) for consistency between transmission and reception.

4.2 Convolutional Encoder (`convEncoder.m`)

The convolutional encoder implements a rate-1/2 code with K=7 constraint length:

- Generator polynomials: [171 133] (octal)
- Constraint length: $K = 7$
- Code rate: 1/2
- Input: 272 information bits
- Output: 544 encoded bits

The encoder uses a shift register implementation where each input bit produces two output bits by computing the parity of specific tap combinations.

4.3 Gaussian Filter (`GaussianFilter.m`)

The Gaussian filter performs pulse shaping to limit spectral occupancy:

- BT product: 0.35 (typical for GMSK)
- Samples per symbol (SPS): 4
- Filter span: 8 symbols

The Gaussian filter shape is characterized by the BT (bandwidth-time) product, which determines the tradeoff between spectral efficiency and intersymbol interference.

4.4 GMSK Mapper (`GmskMapper.m`)

The GMSK mapper converts filtered bits to complex baseband signals:

- LUT size: 512 entries
- Accumulator bits: 16
- Bit gain reduction: 8
- Quantization: Q15 format

The mapper implements phase accumulation where each bit advances the carrier phase by $\pi/2$ radians, producing the characteristic GMSK constant-envelope signal.

5 Receiver Components

5.1 RxDemodulator (`RxDemodulator.m`)

The receiver demodulator converts the received complex signal to soft metrics:

- Samples per symbol: 4 (must match transmitter)
- Gaussian filter span: 8 symbols
- Output: Log-likelihood ratios (LLRs) for each bit

The demodulator applies matched filtering and samples at symbol intervals, accounting for the group delay introduced by the transmitter filter.

5.2 GMSK Demapper (`GmskDemapper.m`)

The demapper extracts phase information from the received signal:

- Computes phase difference between consecutive samples
- Unwraps phase to recover continuous phase trajectory
- Outputs soft metrics proportional to bit reliability

Phase-based demodulation is optimal for GMSK since the information is encoded in phase changes.

5.3 Viterbi Decoder (`ViterbiDecoder.m`)

The Viterbi decoder performs soft-decision convolutional decoding:

- Constraint length: $K = 7$
- Generator polynomials: [171 133] octal
- Traceback length: 35
- Input type: Soft LLR
- Soft bit width: 3 bits
- Mode: Truncated

The decoder converts LLRs to soft metrics using the convention that metric 0 indicates strong '0' and metric $2^N - 1$ indicates strong '1'.

Key parameters:

- `nVar`: Noise variance for LLR scaling
- `llrClip`: LLR clipping range (default 127)
- `tracebackLen`: Number of trellis steps to trace back

6 Packet Detection and Processing

6.1 Preamble Detection (`detect_preamble.m`)

The preamble detector uses normalized cross-correlation:

1. Compute correlation between received LLR stream and known preamble sequence
2. Normalize correlation by preamble length
3. Apply threshold (typically 0.6) to identify peaks
4. Apply non-maximum suppression with minimum spacing (e.g., 500 bits)
5. Return ordered list of detections with indices and correlation values

The detection threshold controls the tradeoff between sensitivity and false alarm rate. Lower thresholds detect more packets but may produce false positives.

6.2 Packet Building (`build_packet.m`)

The packet builder constructs complete bursts:

1. Convert payload bytes to bits (MSB-first)
2. Apply convolutional encoding ($272 \rightarrow 544$ bits)
3. Insert preamble at the start
4. Interleave payload chunks with training sequences
5. Output complete 624-bit burst

The interleaved structure ensures training sequences are distributed throughout the packet for robust channel estimation.

6.3 Payload Extraction (`extract_payload.m`)

The payload extractor removes overhead:

1. Skip preamble bits
2. Extract 4 payload chunks, discarding training sequences
3. Concatenate chunks into continuous encoded payload
4. Output 544 encoded bits

7 Multi-Packet Processing

The system supports transmission of arbitrary-length data by splitting across multiple packets.

7.1 Multi-Packet Builder (`build_multipacket.m`)

1. Read all input bytes from CSV
2. Split into 34-byte chunks (last chunk may be padded)
3. Build each packet using `build_packet`
4. Insert configurable guard bits between packets
5. Return concatenated bitstream with packet metadata

Default guard bits: 100 bits between packets to ensure separation.

7.2 Multi-Packet Reception (`receive_multipacket.m`)

1. For each detection from preamble detector:
 - Extract burst from LLR stream
 - Remove preamble and training sequences
 - Apply Viterbi decoding
 - Convert bits to bytes
2. Concatenate all recovered packet bytes
3. Trim padding from final packet
4. Return aggregate bytes with per-packet status

The receiver handles partial packet failures gracefully, continuing to process detected packets even if some fail.

8 CSV Interface

The system uses AXI4-Stream format for CSV input/output:

8.1 Input Format (`read_axi_stream_csv.m`)

```
TDATA,TKEEP,TLAST
9B553F84,f,0
0789DC7F,f,0
CA04CA04,3,1
```

- TDATA: 32-bit word in hexadecimal (8 hex digits)
- TKEEP: Byte enable mask (f = all 4 bytes valid, 3 = upper 2 bytes)
- TLAST: End of frame flag (1 on last word)

Byte ordering: TDATA stores bytes in big-endian format [B3 B2 B1 B0] where B3 is MSB.

8.2 Output Format (`write_axi_stream_csv.m`)

Same format as input, with decoded bytes written in TDATA field.

9 Test Scripts and Validation

9.1 Single-Packet Test (`test_complete_system.m`)

Tests end-to-end transmission of a single 34-byte packet:

- Eb/N0: 10 dB
- SPS: 4
- BT: 0.35
- Preamble threshold: 0.65
- Generates 9 diagnostic plots including constellation, correlation, and error analysis

Output metrics:

- Byte Error Rate (BER)
- Bit Error Rate
- Error locations
- Transmitted vs received byte comparison

9.2 Multi-Packet Test (`test_multipacket_system.m`)

Tests continuous multi-packet transmission:

- Eb/N0: 30 dB
- SPS: 4
- BT: 0.35
- Preamble threshold: 0.6
- Guard bits: 100
- Minimum packet spacing: 500 bits
- Generates 12 diagnostic plots

Output metrics:

- Number of transmitted/detected/recovered packets
- Packet success rate
- Byte Error Rate
- Bit Error Rate
- Per-packet status
- Detection timeline

9.3 Simple Example (`simple_multipacket_example.m`)

Lightweight demonstration script for quick testing.

10 Performance Parameters

10.1 System Parameters

Parameter	Typical Value	Description
Eb/N0	10-30 dB	Energy per bit to noise spectral density
SPS	4	Samples per symbol
BT	0.35	Gaussian filter bandwidth-time product
Preamble Threshold	0.6-0.65	Correlation detection threshold
Guard Bits	100	Bits between packets in multi-packet mode
Min Packet Spacing	500 bits	Minimum detection spacing for NMS

10.2 Expected Performance

Eb/N0 (dB)	Expected BER	Comments
3	0.05-0.1	Marginal performance
10	0.001-0.01	Usable for most applications
30	0.0001	Excellent performance, near error-free

11 File Inventory

11.1 Core Communication Blocks

File	Description
<code>convEncoder.m</code>	Rate 1/2 convolutional encoder (K=7)
<code>ViterbiDecoder.m</code>	Soft-decision Viterbi decoder
<code>GaussianFilter.m</code>	Gaussian pulse shaping filter
<code>GmskMapper.m</code>	GMSK modulator with LUT
<code>GmskDemapper.m</code>	Phase detection and unwrapping
<code>RxDemodulator.m</code>	Complete demodulator with LLR output

11.2 Packet Processing

File	Description
<code>Waveform.m</code>	Defines packet structure parameters
<code>build_packet.m</code>	Constructs single packet from payload

File	Description
build_multipacket.m	Builds multi-packet transmission stream
extract_payload.m	Extracts payload from received burst
detect_preamble.m	Correlation-based preamble detection
receive_multipacket.m	Processes multiple detected packets

11.3 CSV Interface

File	Description
read_axi_stream_csv.m	Reads AXI4-Stream format CSV
write_axi_stream_csv.m	Writes AXI4-Stream format CSV

11.4 Test Scripts

File	Description
test_complete_system.m	Single-packet end-to-end test
test_multipacket_system.m	Multi-packet end-to-end test
simple_multipacket_example.m	Simple weight demonstration
compare_multipacket_results.m	Diagnostic visualization utility

12 Usage Example

12.1 Running the System

```
% Single packet test
test_complete_system
```

```
% Multi-packet test
test_multipacket_system
```

These scripts will:

1. Read input data from CSV (tx_data.csv or tx_multipacket_data.csv)
2. Build and modulate packets
3. Add AWGN channel noise
4. Demodulate and detect preambles
5. Decode payloads with Viterbi algorithm
6. Write recovered data to output CSV
7. Display diagnostic plots and error statistics

13 Conclusion

This project implements a complete GMSK communication system with robust error correction through convolutional coding and Viterbi decoding. The multi-packet support enables transmission of arbitrary-length data streams, while the CSV interface provides simple integration with external systems using the AXI4-Stream standard.

Key achievements:

- Complete transmitter and receiver chain implementation
- Support for arbitrary-length payload transmission
- Comprehensive diagnostic and visualization tools
- AXI4-Stream CSV format for easy data input/output
- Configurable parameters for different operating conditions

The system demonstrates reliable communication at moderate to high SNR levels and provides a solid foundation for further development such as higher-order modulation, channel estimation improvements, or hardware implementation.